

Lesson 4 — Sparse Matrix Addition Algorithm

1. Where We Are in the Course

Before Lesson 4, you already learned how sparse matrices are stored and displayed.

Lesson 1: Sparse Matrix Foundations

You learned that a sparse matrix is a matrix with mostly zero values. Instead of storing every value, we store only the non-zero values.

Lesson 2: Coordinate List Representation

You learned that each non-zero element is stored using three pieces of information:

```
row, column, value
```

Example:

```
(2, 3, 7)
```

This means:

```
At row 2, column 3, the value is 7.
```

Lesson 3: Displaying a Sparse Matrix

You learned how to display the full matrix even though only the non-zero elements are stored.

The program visits every matrix position. If that position exists in the sparse list, it prints the stored value. Otherwise, it prints `0`.

Lesson 4: Current Lesson

Lesson 4 teaches how to add two sparse matrices using their Coordinate List representations.

The focus is not full C code yet. The main goal is to understand the algorithm clearly before coding it in the next lesson.

2. Main Idea of Sparse Matrix Addition

Adding two normal matrices is done position by position.

For normal matrices:

$$C[i][j] = A[i][j] + B[i][j]$$

Example:

A:		B:		C:
1 0 0		0 5 0		1 5 0
0 7 0	+	0 3 0	=	0 10 0
0 0 2		0 0 0		0 0 2

But in sparse matrices, we do not store all positions.

We only store non-zero values.

So instead of adding every cell in the full matrix, we add the two sparse lists.

In this lesson, the sparse matrices are stored in Coordinate List form.

That means each stored element looks like this:

row, column, value

3. First Rule: Matrix Dimensions Must Match

Two matrices can be added only if they have the same number of rows and columns.

Valid Example

A is 3 x 4
B is 3 x 4

These can be added because both matrices have:

3 rows and 4 columns

Invalid Example

A is 3 x 4
B is 4 x 3

These cannot be added because their sizes are different.

Matrix addition works position by position. So every position in **A** must have a matching position in **B**.

If dimensions do not match, addition is not possible.

Dimension Check Logic

```
if (A.m != B.m || A.n != B.n) {  
    // Addition is not possible  
}
```

Where:

m = number of rows
n = number of columns

So the condition for addition is:

A.m == B.m
A.n == B.n

4. Important Variables

In sparse matrix addition, we commonly use these variables:

A = first sparse matrix
B = second sparse matrix
C = result sparse matrix

Each matrix has:

```
m   = number of rows
n   = number of columns
num = number of non-zero elements
```

Each stored element has:

```
row
column
value
```

In C-style structures, these may be written as:

```
i = row index
j = column index
x = value
```

So an element may look like:

```
struct Element {
    int i; // row
    int j; // column
    int x; // value
};
```

5. Result Matrix Size

When adding two sparse matrices, we do not know the final number of non-zero elements immediately.

So we first allocate enough space for the worst case.

Worst Case

The worst case happens when no positions overlap between **A** and **B**.

Example:

A has non-zero positions:
(0,0), (1,2), (2,2)

B has non-zero positions:
(0,1), (1,0), (2,0), (2,1)

No positions are the same.

So all non-zero elements from both matrices appear in the result.

If:

A.num = 3
B.num = 4

Then the maximum result size is:

A.num + B.num = 3 + 4 = 7

So we allocate result space of size:

A.num + B.num

When the Result Is Smaller

If both matrices contain a non-zero value at the same position, those two values are added and stored as one result element.

Example:

A has: (1,1,7)
B has: (1,1,3)

Both are at the same position:

row = 1
column = 1

So we add the values:

```
7 + 3 = 10
```

The result stores only one entry:

```
(1,1,10)
```

Not two entries.

So the actual result size may be smaller than:

```
A.num + B.num
```

This is why we use a variable `k` to count the actual number of result elements.

At the end:

```
C.num = k
```

6. The Three Pointers: i, j, and k

Sparse matrix addition uses three index variables:

```
i = current element in A  
j = current element in B  
k = current position in C
```

Think of them like three fingers:

```
i points to the current non-zero element in A  
j points to the current non-zero element in B  
k points to the place where we write in C
```

At the beginning:

```
i = 0  
j = 0  
k = 0
```

This means:

```
Start at the first element of A.  
Start at the first element of B.  
Start writing at the first position of C.
```

After copying or adding an element, the pointers move forward.

7. Important Assumption: Row-Major Order

For this algorithm to work correctly and efficiently, both sparse matrices should be stored in row-major order.

Row-Major Order Meaning

Row-major order means:

```
Sort by row first.  
If rows are equal, sort by column.
```

Good Order

```
(0,1,5)  
(0,3,9)  
(2,0,4)  
(3,2,8)
```

This is good because:

- Row `0` comes before row `2`.
- Inside row `0`, column `1` comes before column `3`.

Bad Order

```
(3,2,8)
(0,1,5)
(2,0,4)
(0,3,9)
```

This is bad because the rows are not arranged from top to bottom.

Why Order Matters

The addition algorithm moves forward only.

It does not go backward to search for earlier elements.

So if the elements are not sorted, the algorithm may produce the wrong result.

8. Core Idea: Merge Two Sorted Sparse Lists

Sparse matrix addition is like merging two sorted lists.

At every step, compare:

```
A[i]
B[j]
```

Each element has:

```
row, column, value
```

So we compare positions.

First compare rows:

```
A[i].row with B[j].row
```

If the rows are equal, compare columns:


```
A[i].column with B[j].column
```

The smaller position is copied into the result.

If the positions are the same, the values are added.

9. Merge Rules

Rule 1: If A Row Is Smaller

Example:

```
A[i] = (0, 2, 5)
B[j] = (1, 0, 7)
```

Compare rows:

```
0 < 1
```

So `A[i]` comes first.

Copy `A[i]` into `C[k]`:

```
C[k] = A[i]
```

Then move:

```
i++
k++
```

Meaning:

```
Move to the next element in A.
Move to the next writing position in C.
```

Rule 2: If B Row Is Smaller

Example:

```
A[i] = (2, 1, 6)
B[j] = (1, 3, 4)
```

Compare rows:

```
2 > 1
```

So `B[j]` comes first.

Copy `B[j]` into `C[k]` :

```
C[k] = B[j]
```

Then move:

```
j++
k++
```

Meaning:

```
Move to the next element in B.
Move to the next writing position in C.
```

Rule 3: Rows Equal, A Column Is Smaller

Example:

```
A[i] = (2, 1, 6)
B[j] = (2, 3, 4)
```

Rows are equal:

```
2 == 2
```

Now compare columns:

```
1 < 3
```

So `A[i]` comes first.

Copy `A[i]` into `C[k]`:

```
C[k] = A[i]
```

Then move:

```
i++  
k++
```

Rule 4: Rows Equal, B Column Is Smaller

Example:

```
A[i] = (2, 4, 6)  
B[j] = (2, 3, 4)
```

Rows are equal:

```
2 == 2
```

Compare columns:

```
4 > 3
```

So `B[j]` comes first.

Copy `B[j]` into `C[k]`:

```
C[k] = B[j]
```

Then move:

```
j++  
k++
```

Rule 5: Same Row and Same Column

This is the actual addition case.

Example:

```
A[i] = (2, 2, 7)  
B[j] = (2, 2, 3)
```

Both are at the same position:

```
row = 2  
column = 2
```

So add the values:

```
7 + 3 = 10
```

Store one result element:

```
C[k] = (2, 2, 10)
```

Then move all three pointers:

```
i++  
j++  
k++
```

Why move both `i` and `j`?

Because we used one element from `A` and one element from `B`.

Why move `k`?

Because we wrote one new element into `C`.

10. Merge Rules Summary Table

Condition	Action	Pointer Movement
<code>A row < B row</code>	Copy <code>A[i]</code> into <code>C[k]</code>	<code>i++</code> , <code>k++</code>
<code>A row > B row</code>	Copy <code>B[j]</code> into <code>C[k]</code>	<code>j++</code> , <code>k++</code>
Rows equal and <code>A col < B col</code>	Copy <code>A[i]</code> into <code>C[k]</code>	<code>i++</code> , <code>k++</code>
Rows equal and <code>A col > B col</code>	Copy <code>B[j]</code> into <code>C[k]</code>	<code>j++</code> , <code>k++</code>
Same row and same column	Add values and store once	<code>i++</code> , <code>j++</code> , <code>k++</code>
One list ends	Copy leftovers from the other list	Move source pointer and <code>k</code>

11. Full Dry Run

Now we will trace the algorithm step by step.

Let matrix `A` be stored as:

```
A:
Index  Row  Col  Value
0      0   1    5
1      1   2    7
2      2   2    4
```

Let matrix `B` be stored as:

```
B:
Index  Row  Col  Value
0      0   0    3
1      1   2    6
2      2   3    8
```

Both matrices are `3 x 4`.

So addition is allowed.

Initial values:

```
i = 0
j = 0
k = 0
```

Step 1

Current elements:

```
A[i] = A[0] = (0,1,5)
B[j] = B[0] = (0,0,3)
```

Compare rows:

```
0 == 0
```

Rows are equal, so compare columns:

```
A column = 1
B column = 0
```

Since:

```
0 < 1
```

`B[j]` comes first.

Copy `B[j]` into `C[k]`:

```
C[0] = (0,0,3)
```

Move:

```
j = 1  
k = 1
```

Current result:

```
C:  
0 0 3
```

Step 2

Current elements:

```
A[i] = A[0] = (0,1,5)  
B[j] = B[1] = (1,2,6)
```

Compare rows:

```
0 < 1
```

`A[i]` comes first.

Copy `A[i]` into `C[k]` :

```
C[1] = (0,1,5)
```

Move:

```
i = 1  
k = 2
```

Current result:

```
C:  
0 0 3  
0 1 5
```

Step 3

Current elements:

```
A[i] = A[1] = (1,2,7)
B[j] = B[1] = (1,2,6)
```

Compare rows:

```
1 == 1
```

Compare columns:

```
2 == 2
```

Same row and same column.

So add the values:

```
7 + 6 = 13
```

Store:

```
C[2] = (1,2,13)
```

Move:

```
i = 2
j = 2
k = 3
```

Current result:

```
C:
0 0 3
0 1 5
1 2 13
```


Step 4

Current elements:

```
A[i] = A[2] = (2,2,4)
B[j] = B[2] = (2,3,8)
```

Compare rows:

```
2 == 2
```

Rows are equal, so compare columns:

```
2 < 3
```

`A[i]` comes first.

Copy `A[i]` into `C[k]`:

```
C[3] = (2,2,4)
```

Move:

```
i = 3
k = 4
```

Now `i` has reached the end of `A` because `A.num = 3`.

Current result:

```
C:
0 0 3
0 1 5
1 2 13
2 2 4
```

Step 5: Copy Leftovers

The main comparison loop stops because **A** is finished.

But **B** still has one remaining element:

```
B[j] = B[2] = (2,3,8)
```

So copy this leftover element into **C[k]** :

```
C[4] = (2,3,8)
```

Move:

```
j = 3  
k = 5
```

Now both lists are finished.

12. Final Result of the Dry Run

The final result matrix **C** is:

```
C:  
Index  Row  Col  Value  
0      0    0    3  
1      0    1    5  
2      1    2   13  
3      2    2    4  
4      2    3    8
```

So:

```
C.num = k = 5
```

Important:

```
A.num = 3  
B.num = 3  
A.num + B.num = 6
```

But:

```
C.num = 5
```

Why?

Because one pair overlapped:

```
(1,2,7) + (1,2,6) = (1,2,13)
```

Two entries became one entry.

13. Full Matrix View of the Dry Run

The sparse lists represent these full matrices:

Matrix A

```
0 5 0 0  
0 0 7 0  
0 0 4 0
```

Matrix B

```
3 0 0 0  
0 0 6 0  
0 0 0 8
```

Matrix C = A + B

```
3 5 0 0
0 0 13 0
0 0 4 8
```

The Coordinate List result is:

```
0 0 3
0 1 5
1 2 13
2 2 4
2 3 8
```

14. Beginner-Friendly Pseudocode

This is not the full C program yet. It is the algorithm in simple form.

```
if dimensions are different:
    addition is not possible

create result array with size A.num + B.num

i = 0
j = 0
k = 0

while i < A.num and j < B.num:

    if A[i].row < B[j].row:
        copy A[i] to C[k]
        i++
        k++

    else if A[i].row > B[j].row:
        copy B[j] to C[k]
        j++
        k++

    else:
        // rows are equal, so compare columns
```

```

        if A[i].column < B[j].column:
            copy A[i] to C[k]
            i++
            k++

        else if A[i].column > B[j].column:
            copy B[j] to C[k]
            j++
            k++

        else:
            // same row and same column
            C[k].row = A[i].row
            C[k].column = A[i].column
            C[k].value = A[i].value + B[j].value
            i++
            j++
            k++

copy remaining elements from A, if any
copy remaining elements from B, if any

C.num = k

```

15. C-Style Algorithm Skeleton

This is closer to C logic, but still mainly for understanding.

```

if (A.m != B.m || A.n != B.n) {
    // addition not possible
}

C.m = A.m;
C.n = A.n;
C.e = allocate space for A.num + B.num elements;

i = 0;
j = 0;
k = 0;

while (i < A.num && j < B.num) {

    if (A.e[i].i < B.e[j].i) {

```

```

        C.e[k++] = A.e[i++];
    }
    else if (A.e[i].i > B.e[j].i) {
        C.e[k++] = B.e[j++];
    }
    else {
        if (A.e[i].j < B.e[j].j) {
            C.e[k++] = A.e[i++];
        }
        else if (A.e[i].j > B.e[j].j) {
            C.e[k++] = B.e[j++];
        }
        else {
            C.e[k].i = A.e[i].i;
            C.e[k].j = A.e[i].j;
            C.e[k].x = A.e[i].x + B.e[j].x;
            i++;
            j++;
            k++;
        }
    }
}

while (i < A.num) {
    C.e[k++] = A.e[i++];
}

while (j < B.num) {
    C.e[k++] = B.e[j++];
}

C.num = k;

```

16. Why We Copy Leftovers

The main loop runs only while both matrices still have elements.

That condition is:

```
i < A.num and j < B.num
```

If one matrix finishes first, the loop stops.

But the other matrix may still have remaining elements.

Those remaining elements are called leftovers.

Example

A:
(0,0,5)

B:
(0,1,7)
(2,3,9)

After processing `A`, matrix `A` is finished.

But `B` still has:

(2,3,9)

This value must be copied into the result.

Why?

Because there is no matching value in `A`, so it remains unchanged in the sum.

So after the main loop, we do:

```
copy remaining elements from A, if any  
copy remaining elements from B, if any
```

17. Why `C.num = k`

At the start, we allocate space for:

`A.num + B.num`

But this is only the maximum possible size.

The actual number of elements stored in `C` is counted by `k`.

Every time we write one result element, we do:

```
k++
```

So at the end:

```
k = actual number of non-zero elements in C
```

Therefore:

```
C.num = k
```

Do not set:

```
C.num = A.num + B.num
```

That may be wrong when some positions overlap.

18. Why This Algorithm Is Efficient

A normal full matrix addition checks every position.

If the matrix has:

```
m rows  
n columns
```

then a full matrix has:

```
m * n positions
```

So normal matrix addition takes:

```
O(m * n)
```

But sparse matrix addition only checks stored non-zero elements.

If:

```
A.num = number of non-zero elements in A  
B.num = number of non-zero elements in B
```

then sparse addition scans at most:

```
A.num + B.num
```

So the time complexity is:

```
O(A.num + B.num)
```

This is efficient when the matrices are very sparse.

19. Time Complexity

The algorithm scans each non-zero element at most once.

The pointer `i` only moves forward through `A`.

The pointer `j` only moves forward through `B`.

No element is repeatedly searched.

So the time complexity is:

```
O(A.num + B.num)
```

If we call the number of non-zero elements:

```
num1 = A.num  
num2 = B.num
```

then the time complexity can also be written as:

```
O(num1 + num2)
```

20. Space Complexity

The result matrix may need to store all elements from both matrices.

This happens when no coordinates overlap.

So result space is:

```
O(A.num + B.num)
```

or:

```
O(num1 + num2)
```

This is the worst-case result storage.

21. Common Beginner Mistakes

Mistake 1: Forgetting the Dimension Check

Wrong idea:

```
Add matrices even if their sizes are different.
```

Correct idea:

```
Only add if A.m == B.m and A.n == B.n.
```

Mistake 2: Comparing Columns Before Rows

Wrong order:

```
Compare column first.
```

Correct order:

```
Compare row first.  
If rows are equal, compare column.
```

Why?

Because the sparse elements are sorted in row-major order.

Mistake 3: Moving Only One Pointer When Coordinates Match

Wrong:

```
same coordinate:  
add values  
i++  
k++
```

Correct:

```
same coordinate:  
add values  
i++  
j++  
k++
```

Why?

Because one element from **A** and one element from **B** were both used.

Mistake 4: Forgetting to Copy Leftovers

Wrong idea:

```
Stop when one matrix finishes.
```

Correct idea:

```
After one matrix finishes, copy all remaining elements from the other matrix.
```

Mistake 5: Setting Result Count Incorrectly

Wrong:

```
C.num = A.num + B.num
```

Correct:

```
C.num = k
```

Why?

Because `k` stores the actual number of elements written into the result.

Mistake 6: Forgetting Sorted Order

Wrong idea:

```
The elements can be stored in any random order.
```

Correct idea:

```
For this merge algorithm, elements should be stored in row-major order.
```

22. Simple Mental Model

Imagine two lists of important locations.

Each location has:

```
row, column, value
```

You compare the first location from each list.

- If one location comes earlier, copy it.
- If both locations are the same, add their values.
- Then move forward.

This is like merging two sorted lists.

The result is also a sorted list.

23. Mini Practice Example

Try tracing this:

```
A:
Row  Col  Value
0     0     4
1     2     5
2     1     7
```

```
B:
Row  Col  Value
0     0     6
1     1     3
2     1     8
```

Both matrices have the same dimensions.

Initial pointers:

```
i = 0
j = 0
k = 0
```

Practice Trace

Step 1

```
A[i] = (0,0,4)
B[j] = (0,0,6)
```

Same row and same column.

Add:

$4 + 6 = 10$

Store:

$C[0] = (0, 0, 10)$

Move:

$i++$
 $j++$
 $k++$

Step 2

$A[i] = (1, 2, 5)$
 $B[j] = (1, 1, 3)$

Rows are equal.

Compare columns:

$2 > 1$

So copy $B[j]$:

$C[1] = (1, 1, 3)$

Move:

$j++$
 $k++$

Step 3

```
A[i] = (1,2,5)
B[j] = (2,1,8)
```

Compare rows:

```
1 < 2
```

So copy `A[i]`:

```
C[2] = (1,2,5)
```

Move:

```
i++
k++
```

Step 4

```
A[i] = (2,1,7)
B[j] = (2,1,8)
```

Same row and same column.

Add:

```
7 + 8 = 15
```

Store:

```
C[3] = (2,1,15)
```

Move:

```
i++  
j++  
k++
```

Both lists are finished.

Practice Final Answer

```
C:  
Row   Col   Value  
0      0     10  
1      1      3  
1      2      5  
2      1     15
```

Explanation:

```
(0,0): 4 + 6 = 10  
(1,1): only in B, so copy 3  
(1,2): only in A, so copy 5  
(2,1): 7 + 8 = 15
```

24. Quick Self-Check Questions

Question 1

When can two sparse matrices be added?

Answer

Only when they have the same number of rows and columns.

```
A.m == B.m  
A.n == B.n
```

Question 2

Why do we allocate result space as `A.num + B.num`?

Answer

Because in the worst case, no coordinates overlap, so the result may need to store all non-zero elements from both matrices.

Question 3

What does `i` do?

Answer

`i` points to the current element in matrix `A`.

Question 4

What does `j` do?

Answer

`j` points to the current element in matrix `B`.

Question 5

What does `k` do?

Answer

`k` points to the current writing position in result matrix `C`.

Question 6

What happens if `A[i]` and `B[j]` have the same row and same column?

Answer

Their values are added and stored as one result element.

Question 7

Why do we set `C.num = k` at the end?

Answer

Because `k` is the actual number of result elements written.

Question 8

What is the time complexity of sparse matrix addition?

Answer

`O(A.num + B.num)`

or:

`O(num1 + num2)`

Question 9

What is the worst-case result space complexity?

Answer

`O(A.num + B.num)`

or:

`O(num1 + num2)`

Question 10

Why should the input elements be in row-major order?

Answer

Because the algorithm moves forward through the arrays only. Sorted order allows it to compare and merge efficiently.

25. Lesson 4 Final Summary

Sparse matrix addition uses a merge-style algorithm.

The main steps are:

1. Check that both matrices have the same dimensions.
2. Allocate result space of size $A.\text{num} + B.\text{num}$.
3. Start $i = 0$, $j = 0$, $k = 0$.
4. Compare $A[i]$ and $B[j]$.
5. Compare rows first.
6. If rows are equal, compare columns.
7. If one coordinate comes first, copy it to C .
8. If both coordinates are the same, add the values.
9. Move the correct pointers.
10. Copy leftovers after one list ends.
11. Set $C.\text{num} = k$.

The most important idea:

Sparse matrix addition does not add every cell.
It only scans the stored non-zero elements.

That is why the time complexity is:

$O(A.\text{num} + B.\text{num})$

not:

$O(m * n)$

Lesson 4 prepares you for Lesson 5, where this algorithm becomes a complete sparse matrix addition program.